

Software Requirements Specification

PotionCheck - AI Ingredient Intelligence Scanner

Version: 1.0
Date: 20 May 2026
Prepared for: PBL 6 Project
Prepared by: Project Team
Template basis: IEEE-style Software Requirements Specification

Revision History

Version	Date	Description	Author
1.0	20 May 2026	Initial SRS for PotionCheck web application	Project Team

Table of Contents

Revision Historyii

1. Introduction1

1.1 Purpose2

1.2 Document Conventions2

1.3 Intended Audience and Reading Suggestions2

1.4 Product Scope3

1.5 References3

2. Overall Description4

2.1 Product Perspective4

2.2 Product Functions4

2.3 User Classes and Characteristics4

2.4 Operating Environment5

2.5 Design and Implementation Constraints5

2.6 User Documentation5

2.7 Assumptions and Dependencies6

3. System Features6

3.1 Authentication and User Session Management6

3.2 Health Profile Management7

3.3 Barcode Scanning and Product Lookup7

3.4 Label Upload and OCR8

3.5 Pasted Ingredient Text Analysis8

3.6 AI Ingredient and Nutrition Analysis9

3.7 Analysis Report Display10

3.8 Scan History10

3.9 System Health Monitoring11

4. External Interface Requirements11

4.1 User Interfaces11

4.2 Hardware Interfaces12

4.3 Software Interfaces12

4.4 Communication Interfaces13

5. Non-Functional Requirements14

5.1 Performance Requirements14

5.2 Reliability and Availability14

5.3 Security Requirements14

5.4 Usability Requirements15

5.5 Maintainability Requirements15

5.6 Portability Requirements15

6. Data Requirements15

6.1 Data Model15

6.2 Data Validation16

6.3 Data Retention16

7. Other Requirements17

7.1 Deployment Requirements17

7.2 AI Provider Behavior18

7.3 Safety and Disclaimer Requirement18

7.4 Known Limitations18

7.5 Future Enhancements18

8. Appendices18

Appendix A: Main Technology Stack18

Appendix B: Verdict Definitions19

Appendix C: Requirement Traceability Summary19

1. Introduction

1.1 Purpose

This Software Requirements Specification describes the software requirements for PotionCheck, an AI-powered ingredient intelligence scanner. PotionCheck is a full-stack web application that helps users understand packaged food products by scanning barcodes, uploading label images, pasting ingredient text, or selecting fresh produce.

The purpose of this document is to define the functional requirements, non-functional requirements, interface requirements, data requirements, system constraints, and deployment expectations for the PotionCheck project. It covers both major subsystems of the application: the React/Vite frontend used by consumers and the FastAPI backend used for authentication, product lookup, OCR processing, AI analysis, scan history, and health monitoring.

This SRS will be used as a reference by developers, testers, project evaluators, and maintainers to understand what the system must do, how users interact with it, and how the application should behave under normal and fallback conditions.

1.2 Document Conventions

This document follows an IEEE-style Software Requirements Specification format. Section numbers are used to organize the document into introduction, overall description, system features, interface requirements, non-functional requirements, data requirements, and appendices.

The following conventions are used:

- Functional requirements are labeled as FR-001, FR-002, and so on.
- Non-functional requirements are labeled as NFR-001, NFR-002, and so on.
- Backend API endpoints are written in monospace format, such as `/api/scanner/barcode/analyze`.
- File paths, configuration keys, and commands are written in monospace format.
- The words "shall" and "must" indicate mandatory requirements.
- The words "should" and "may" indicate recommended or optional behavior.
- Requirements in the same table are considered equal priority unless a future project plan assigns priority values.

1.3 Intended Audience and Reading Suggestions

This document is intended for the following readers:

- Project evaluators and faculty members who need to verify the scope, requirements, and implementation direction of the PBL 6 project.
- Developers who will implement or maintain the React frontend, FastAPI backend, AI analysis service, barcode scanner, OCR workflow, and deployment configuration.
- Testers who need to prepare test cases for authentication, profile management, barcode analysis, OCR upload, pasted-text analysis, AI fallback behavior, history, and error handling.
- Deployment maintainers who need to configure Docker Compose, Render, environment variables, database storage, Redis, Groq, Ollama, and CORS settings.
- Future contributors who need a structured overview of the system before modifying or extending it.

Suggested reading order:

1. Read Section 1 to understand the purpose, scope, and references.

2. Read Section 2 for the overall system context, users, assumptions, and constraints.
3. Read Section 3 for detailed functional requirements.
4. Read Section 4 for frontend, backend, hardware, software, and API interface requirements.
5. Read Section 5 for performance, reliability, security, usability, maintainability, and portability requirements.
6. Read Section 6 for database and data validation details.
7. Read Section 7 and the appendices for deployment notes, limitations, and requirement traceability.

1.4 Product Scope

PotionCheck is designed to help users make better food choices by converting product labels and nutrition data into understandable, personalized ingredient reports. The system combines barcode product lookup, OCR, AI reasoning, user health profiles, and scan history in a single web application.

The product supports the following main capabilities:

- Provide quick product analysis from barcodes, uploaded labels, pasted text, and produce selection.
- Detect potentially harmful or caution-worthy ingredients.
- Personalize warnings using allergies, health conditions, and diet type.
- Show health score, safety verdict, nutrition breakdown, AI summary, recommendation, frequency advice, and ingredient-level explanations.
- Generate a natural "Why this matters for you" explanation based on the user's profile and food-science context.
- Preserve scan history for authenticated users.
- Use Groq for primary LLM analysis and Ollama as fallback when Groq is unavailable or usage limits are exceeded.
- Continue functioning with local rule-based analysis when external or local AI services are unavailable.

The project is not intended to replace medical advice, diagnosis, or professional nutrition consultation. It provides educational and informational guidance based on available product data, user profile inputs, and AI/rule-based analysis.

1.5 References

The following documents, files, and resources are referenced by this SRS:

- IEEE-style SRS template: `C:\Users\LOQ\Downloads\srs_template-ieee.pdf`
- Project deployment guide: `DEPLOYMENT.md`
- Docker Compose configuration: `docker-compose.yml`
- Render deployment configuration: `render.yaml`
- Frontend package definition: `frontend/package.json`
- Backend dependencies: `backend/requirements.txt`
- Frontend routing and page structure: `frontend/src/App.jsx`
- Backend application entry point: `backend/app/main.py`
- Backend API routers: `backend/app/api/`
- Backend database models: `backend/app/models/entities.py`
- AI analysis service: `backend/app/services/ai.py`
- Open Food Facts product lookup service: `backend/app/services/open_food_facts.py`
- Open Food Facts API: <https://world.openfoodfacts.org>
- Groq API for LLM analysis

- Ollama local model runtime for fallback LLM analysis

2. Overall Description

2.1 Product Perspective

PotionCheck is an independent web-based health and ingredient analysis system. It consists of:

- A React single-page frontend built with Vite and Tailwind CSS.
- A FastAPI backend that exposes REST APIs.
- SQLite database storage through SQLAlchemy async models.
- Redis caching for product lookup data.
- Open Food Facts integration for barcode product information.
- Groq LLM integration for AI analysis with Ollama fallback.
- OCR support through Tesseract for uploaded label images.

The application can run locally through separate frontend/backend commands, Docker Compose, or a cloud deployment such as Render.

2.2 Product Functions

The major functions of PotionCheck are:

- User registration, login, logout, token refresh, and profile retrieval.
- Health profile creation and update.
- Barcode scanning through live camera, uploaded barcode image, or manual entry.
- Product data lookup from Open Food Facts.
- Ingredient text extraction from uploaded label images.
- Ingredient analysis from pasted text.
- AI-generated health verdict, safety score, health score, nutrition observations, ingredient explanations, and recommendation.
- Personalized warnings based on allergies, health conditions, and diet type.
- Scan history storage, filtering, searching, and deletion.
- Local frontend state persistence for profile and recent scan data.
- Health check endpoint for backend, database, Redis, and Open Food Facts availability.

2.3 User Classes and Characteristics

User Class	Description	Expected Skill Level
Guest User	Uses scanning and analysis without signing in; local history/profile may be stored in browser state.	Basic web usage
Registered User	Creates account, saves profile, receives personalized analysis, and can access server-side history.	Basic web usage

Health-Conscious Consumer	Uses the product to review packaged food ingredients and nutrition.	Basic nutrition awareness
Developer/Maintainer	Runs, debugs, deploys, and extends the application.	Web development knowledge
Evaluator/Instructor	Reviews the system as a PBL project.	General technical review

2.4 Operating Environment

Frontend environment:

- Browser: Chrome, Edge, or any modern browser with JavaScript enabled.
- Camera access requires localhost or HTTPS browser context.
- Node.js environment for development and build.

Backend environment:

- Python 3 environment.
- FastAPI with Uvicorn.
- SQLite database by default.
- Redis for cache.
- Network access to Open Food Facts.
- Optional network/API access to Groq.
- Optional local Ollama server.

Deployment environment:

- Docker Compose for local multi-service setup.
- Render deployment using `render.yaml`.

2.5 Design and Implementation Constraints

- The frontend is a React single-page application using React Router.
- The backend must expose REST APIs under `/api`.
- Authentication uses JWT access tokens and HTTP-only refresh cookies.
- Passwords must be hashed using `bcrypt`.
- Barcode lookup depends on Open Food Facts availability.
- Groq API use requires `GROQ_API_KEY`.
- Ollama fallback requires a local or reachable Ollama server.
- Camera scanning depends on browser permissions and secure context requirements.
- SQLite is used for persistence; scalability is limited compared with client-server databases.

2.6 User Documentation

The application provides user guidance through interface labels, toasts, form validation messages, scan status messages, and deployment documentation. Additional user documentation may include:

- How to run the app locally.

- How to configure Groq and Ollama.
- How to scan a barcode or upload an ingredient label.
- How to interpret scores and warnings.

2.7 Assumptions and Dependencies

Assumptions:

- Users understand that the application provides informational guidance and not professional medical advice.
- Product data from Open Food Facts may be incomplete or inaccurate.
- AI-generated recommendations may require review and should not be treated as clinical diagnosis.
- Users grant camera permission when using live barcode scanning.

Dependencies:

- Open Food Facts for barcode product data.
- Groq API for primary LLM analysis.
- Ollama for fallback LLM analysis.
- Tesseract OCR for label text extraction.
- Redis for cache.
- SQLite for local/persistent data.

3. System Features

3.1 Authentication and User Session Management

Description

The system shall allow users to register, log in, refresh tokens, log out, and retrieve the current authenticated user.

Functional Requirements

ID	Requirement
FR-001	The system shall allow a new user to register with email, password, and full name.
FR-002	The system shall reject registration when the email is already registered.
FR-003	The system shall validate email format and password length.
FR-004	The system shall allow users to log in with valid credentials.
FR-005	The system shall reject invalid login credentials.

FR-006	The system shall issue JWT access tokens after successful login or registration.
FR-007	The system shall store refresh tokens in HTTP-only cookies.
FR-008	The system shall allow users to refresh access tokens with valid refresh cookies.
FR-009	The system shall allow users to log out and clear refresh cookies.
FR-010	The system shall expose a current-user endpoint for authenticated users.

3.2 Health Profile Management

Description

The system shall store user health profile information for personalized ingredient analysis.

Functional Requirements

ID	Requirement
FR-011	The system shall allow authenticated users to retrieve their saved profile.
FR-012	The system shall allow authenticated users to save allergies, health conditions, and diet type.
FR-013	The system shall validate health profile values against allowed lists.
FR-014	The system shall support allergies such as nuts, gluten, dairy, soy, eggs, shellfish, peanuts, and fish.
FR-015	The system shall support health conditions such as diabetes, hypertension, heart disease, pregnancy, kidney disease, celiac disease, and lactose intolerance.
FR-016	The frontend shall persist a local profile for anonymous or offline-style use.

3.3 Barcode Scanning and Product Lookup

Description

The system shall let users analyze products using live barcode scanning, uploaded barcode images, or manual barcode entry.

Functional Requirements

ID	Requirement
FR-017	The system shall support manual barcode entry for 8 to 13 digit UPC/EAN codes.

FR-018	The system shall validate barcode format before sending analysis requests.
FR-019	The frontend shall support live barcode scanning using browser camera access.
FR-020	The frontend shall support barcode detection from uploaded images.
FR-021	The backend shall fetch product details from Open Food Facts by barcode.
FR-022	The backend shall cache product lookup results.
FR-023	The backend shall return a 404 response when a barcode product is not found.
FR-024	The backend shall return a clear error when Open Food Facts lookup fails.
FR-025	The system shall analyze barcode products when ingredients are available.
FR-026	The system shall notify the user if a product exists but ingredient data is missing.

3.4 Label Upload and OCR

Description

The system shall allow users to upload food label images and extract ingredient text.

Functional Requirements

ID	Requirement
FR-027	The frontend shall allow image upload for product label OCR.
FR-028	The backend shall accept image uploads only.
FR-029	The backend shall reject files larger than 10 MB.
FR-030	The backend shall store uploaded images temporarily in the uploads directory.
FR-031	The backend shall run OCR on uploaded images.
FR-032	The frontend shall display extracted text for user review and editing.
FR-033	The system shall allow analysis of OCR-extracted ingredients.

3.5 Pasted Ingredient Text Analysis

Description

The system shall allow users to paste ingredient text directly and request analysis.

Functional Requirements

ID	Requirement
FR-034	The frontend shall provide a text area for pasted ingredient lists.
FR-035	The frontend shall support basic ingredient parsing for user preview.
FR-036	The backend shall validate that ingredient text is not empty.
FR-037	The backend shall analyze pasted ingredient text without requiring barcode data.

3.6 AI Ingredient and Nutrition Analysis

Description

The system shall generate AI-based ingredient analysis using Groq as the primary provider and Ollama as fallback.

Functional Requirements

ID	Requirement
FR-038	The backend shall analyze ingredients using the configured AI provider.
FR-039	The default AI provider shall be Groq when configured.
FR-040	The backend shall fall back to Ollama when Groq is unavailable, rate-limited, or not configured.
FR-041	The backend shall fall back to local rule-based analysis when Ollama is unavailable.
FR-042	The analysis result shall include safety score, verdict, health score, and health verdict.
FR-043	The analysis result shall include flagged ingredients with severity, reason, scientific name, and personalized warning.
FR-044	The analysis result shall include all ingredients with status, reason, personalized explanation, benefit, excess-use concern, and "why this matters" paragraph.
FR-045	The analysis result shall include sugar analysis, sodium analysis, fitness analysis, daily use advice, and weekly use advice.
FR-046	The analysis result shall include health suitability flags such as healthy, gym friendly, and weight loss friendly.
FR-047	The AI prompt shall require valid JSON output.

FR-048	The backend shall normalize AI output to protect the frontend from missing fields.
--------	--

3.7 Analysis Report Display

Description

The frontend shall display the complete analysis in a user-friendly report page.

Functional Requirements

ID	Requirement
FR-049	The report shall show product name, brand, barcode, categories, and product image when available.
FR-050	The report shall show health score and verdict through a visual meter.
FR-051	The report shall show ingredient cards with expandable details.
FR-052	The ingredient card shall show "Why this matters for you" as one natural paragraph rather than separate labeled output.
FR-053	The report shall show nutriment breakdown for carbohydrates, protein, fat, sugar, sodium, and fiber.
FR-054	The report shall show AI summary and recommendation.
FR-055	The report shall warn when fallback/local rules were used.
FR-056	The frontend shall allow saving analysis to local history.
FR-057	The frontend shall allow sharing a report as an image.

3.8 Scan History

Description

Authenticated users shall be able to retrieve, search, filter, and delete scan history.

Functional Requirements

ID	Requirement
FR-058	The backend shall return paginated scan history for the authenticated user.
FR-059	The backend shall support filtering history by verdict.

FR-060	The backend shall support searching history by product name.
FR-061	The backend shall allow deletion of a single scan.
FR-062	The backend shall allow deletion of all scans only when confirmation matches the user's email.
FR-063	The frontend shall persist recent local history for quick access.

3.9 System Health Monitoring

Description

The backend shall expose service health information.

Functional Requirements

ID	Requirement
FR-064	The backend shall expose <code>/api/health/</code> .
FR-065	The health endpoint shall report database status.
FR-066	The health endpoint shall report Redis status.
FR-067	The health endpoint shall report Open Food Facts availability.
FR-068	The health endpoint shall include a timestamp.

4. External Interface Requirements

4.1 User Interfaces

The frontend shall include the following pages:

Page	Route	Purpose
Landing	<code>/</code>	Introduce the app and route users to features.
Profile	<code>/profile</code>	Register/login and manage health profile.
Scanner	<code>/scanner</code>	Scan barcode, upload label, paste text, or select produce.

Analysis	/analysis/:productId	Display generated ingredient and nutrition report.
History	/history	View saved scan history.
About	/about	Explain application purpose.
Developer	/developer	Present developer/project information.

UI behavior requirements:

- The UI shall show toast notifications for success, warning, and error events.
- The UI shall show progress indicators during analysis and OCR.
- The UI shall be responsive for desktop and mobile screens.
- The UI shall handle network and backend errors with meaningful messages.
- The barcode scanner shall notify users when camera access is unavailable.

4.2 Hardware Interfaces

- Camera: Required for live barcode scanning.
- Keyboard: Required for manual barcode entry and pasted ingredient text.
- File input: Required for uploading label or barcode images.
- Network interface: Required for backend API, Open Food Facts, Groq, and deployment access.

4.3 Software Interfaces

Frontend libraries:

- React 18
- React Router
- Vite
- Tailwind CSS
- Framer Motion
- Zustand
- Axios
- html5-qrcode
- Tesseract.js
- Three.js and related visual libraries

Backend libraries:

- FastAPI
- Uvicorn
- SQLAlchemy async
- Aiosqlite
- Pydantic
- python-jose
- passlib bcrypt
- Redis

- HTTPX
- Groq SDK
- Pillow
- pytesseract

External services:

- Open Food Facts API
- Groq Chat Completions API
- Ollama local chat API
- Redis server

4.4 Communication Interfaces

The frontend communicates with the backend through HTTP/HTTPS REST API calls. JSON is used for request and response bodies except for file upload, which uses multipart form data.

Main backend endpoints:

Method	Endpoint	Purpose
POST	/api/auth/register	Register user
POST	/api/auth/login	Login user
POST	/api/auth/refresh	Refresh access token
POST	/api/auth/logout	Logout user
GET	/api/auth/me	Retrieve current user
GET	/api/profile/	Get user profile
POST	/api/profile/	Save user profile
POST	/api/scanner/barcode	Fetch product by barcode
POST	/api/scanner/barcode/analyze	Analyze barcode product
POST	/api/scanner/upload	Upload image for OCR
POST	/api/scanner/text	Clean and parse ingredient text
POST	/api/analysis/analyze	Analyze custom ingredient payload
GET	/api/analysis/{analysis_id}	Get saved analysis
GET	/api/analysis/product/{barcode}	Get latest analysis for barcode

GET	/api/history/	Get scan history
DELETE	/api/history/{scan_id}	Delete one scan
DELETE	/api/history/all	Delete all scans
GET	/api/health/	Service health check

5. Non-Functional Requirements

5.1 Performance Requirements

ID	Requirement
NFR-001	The frontend shall provide immediate validation feedback for barcode length and format.
NFR-002	Product lookup results should be cached for 24 hours when Redis is available.
NFR-003	The UI shall display progress feedback for long-running OCR and AI analysis operations.
NFR-004	The Ollama request timeout shall be configurable.

5.2 Reliability and Availability

ID	Requirement
NFR-005	The system shall return usable fallback analysis when AI providers are unavailable.
NFR-006	The system shall not crash the frontend when optional AI fields are missing.
NFR-007	The backend shall return structured HTTP errors for invalid input and unavailable product data.
NFR-008	The health endpoint shall help operators identify dependency outages.

5.3 Security Requirements

ID	Requirement
NFR-009	Passwords shall be hashed using bcrypt before storage.
NFR-010	Access tokens shall be JWT-based and time-limited.

NFR-027	The backend shall run as a Docker service.
NFR-028	The complete stack shall be runnable through Docker Compose.
NFR-029	Render deployment shall be supported through <code>render.yaml</code> .

6. Data Requirements

6.1 Data Model

The backend database contains the following main entities:

User

Field	Description
id	UUID string primary key
email	Unique user email
hashed_password	Bcrypt password hash
full_name	User full name
is_active	Account active flag
created_at	Creation timestamp
updated_at	Update timestamp

UserProfile

Field	Description
id	UUID string primary key
user_id	Linked user
allergies	JSON list of allergies
health_conditions	JSON list of health conditions
diet_type	Diet preference

updated_at	Update timestamp
------------	------------------

ScanHistory

Field	Description
id	UUID string primary key
user_id	Optional linked user
barcode	Product barcode or TEXT
product_name	Product/report name
product_image_url	Optional product image
safety_score	Integer score
verdict	SAFE, CAUTION, or DANGER
flagged_count	Number of flagged ingredients
raw_product_data	Raw source product JSON
created_at	Scan timestamp

Analysis

Field	Description
id	UUID string primary key
scan_id	Linked scan
safety_score	Safety score
verdict	Safety verdict
flagged_ingredients	JSON list of flagged ingredients
all_ingredients	JSON list of all ingredient explanations
nutriments	JSON nutrition data
ai_summary	AI summary text

ai_recommendation	AI recommendation text
personalized_warnings	JSON list of warnings
created_at	Analysis timestamp

6.2 Data Validation

- Email must match a valid email pattern.
- Password must be at least 8 characters.
- Barcode must be 8 to 13 digits.
- Ingredient text must not be empty.
- Uploaded file must be an image.
- Uploaded file must not exceed 10 MB.
- Health profile values must match allowed condition and diet lists.

6.3 Data Retention

- Authenticated scan history is stored in the backend database until deleted by the user.
- Local frontend scan history is stored in browser storage and limited to recent items.
- Uploaded OCR files are stored in the backend upload directory and should be cleaned periodically in production.
- Redis product cache entries are intended to expire after 24 hours.

7. Other Requirements

7.1 Deployment Requirements

Local backend:

```
cd backend
python -m uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Local frontend:

```
cd frontend
npm ci
npm run dev
```

Docker Compose:

```
docker compose up --build
```

Production deployment:

- Backend: Docker web service.
- Frontend: Static Vite build.
- Redis: Managed Redis service.
- Required environment variables include `SECRET_KEY`, `AI_PROVIDER`, `GROQ_API_KEY`, `DATABASE_URL`, `REDIS_URL`, `ALLOWED_ORIGINS`, `PRODUCTION_DOMAIN`, and `VITE_API_BASE_URL`.

7.2 AI Provider Behavior

The system shall use the following provider order:

1. Groq for primary LLM analysis when configured.
2. Ollama when Groq fails, exceeds limits, times out, or is unavailable.
3. Local rule-based analysis when Ollama is unavailable.

The AI output shall be normalized before storage and frontend display. The frontend shall not depend on raw provider-specific response formats.

7.3 Safety and Disclaimer Requirement

PotionCheck shall be treated as an educational and informational tool. It shall not claim to diagnose, treat, or prevent disease. Recommendations should encourage users with medical conditions, pregnancy, allergies, or severe dietary restrictions to verify information and seek professional advice when necessary.

7.4 Known Limitations

- Product data depends on Open Food Facts coverage and accuracy.
- OCR accuracy depends on image clarity, label quality, and language.
- Barcode scanning quality depends on camera, lighting, focus, and browser permissions.
- AI output may require review and can vary between providers.
- SQLite is suitable for project-scale deployment but may need migration for high-traffic production.
- Nutrition scoring uses available product data; missing ingredients or nutrients reduce certainty.

7.5 Future Enhancements

- Add PDF export for analysis reports.
- Add admin dashboard for system monitoring.
- Add multilingual OCR and translation.
- Add serving-size based scoring in addition to per-100g scoring.
- Add PostgreSQL support for larger deployments.
- Add scheduled cleanup for uploaded OCR files.
- Add automated backend and frontend test suites.
- Add nutrition source citations in the user report.

8. Appendices

Appendix A: Main Technology Stack

Layer	Technology
Frontend	React, Vite, Tailwind CSS
State	Zustand
HTTP Client	Axios
Animation	Framer Motion, Three.js
Barcode	html5-qrcode
OCR	Tesseract.js frontend, pytesseract backend
Backend	FastAPI, Uvicorn
Database	SQLite with SQLAlchemy async
Cache	Redis
AI	Groq, Ollama, local rules
Deployment	Docker Compose, Render

Appendix B: Verdict Definitions

Verdict	Meaning
SAFE	No major concerns detected in available data; normal portions may be acceptable.
CAUTION	Some ingredients, nutriments, or profile-related concerns require moderation.
DANGER	Significant allergy, ingredient, or nutritional concerns were detected.

Appendix C: Requirement Traceability Summary

Feature Area	Requirement IDs
Authentication	FR-001 to FR-010
Profile	FR-011 to FR-016
Barcode	FR-017 to FR-026
OCR Upload	FR-027 to FR-033

Text Analysis	FR-034 to FR-037
AI Analysis	FR-038 to FR-048
Report UI	FR-049 to FR-057
History	FR-058 to FR-063
Health Monitoring	FR-064 to FR-068
Non-functional	NFR-001 to NFR-029